

Neural Networks Essentials

Recurrent Neural Networks (RNNs)

Denis C. Ilie-Ablachim

April 1, 2026

Lecture Roadmap

Why Sequence Modeling

RNN as a Memory System

Why Vanilla RNNs Fail

LSTM: Fixing the Problem

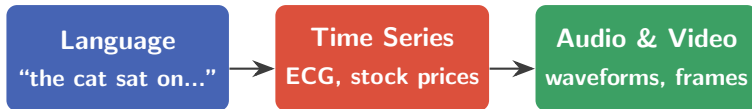
GRU: A Simpler Alternative

Practical Considerations

The Bottleneck and What Comes Next

Summary

Data Has a Time Dimension



Order matters. Context matters.

In all these domains, the meaning of an element depends on **what came before it**.

What Makes a Sequence Different from a Set?

Consider two inputs to a model:

As a set (order ignored)

{dog, the, bit, man, the}

No way to distinguish:

“The dog bit the man” vs.

“The man bit the dog”

As a sequence (order preserved)

[The, dog, bit, the, man]

Now subject, verb, object
are all determined by position.

A model that processes sequences must be sensitive to **position** and **context**—the same token can mean different things depending on where it appears.

Sequence Tasks Are Everywhere

One-to-many

Input: image → Output: caption

“A dog running on a beach”

Many-to-one

Input: movie review → Output: sentiment

“Loved it!” → positive

Many-to-many (aligned)

Input: sentence → Output: POS tags

[The, cat, sat] → [DET, N, V]

Many-to-many (unaligned)

Input: English → Output: French

Different lengths!

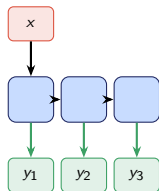
All of these require the model to handle **variable-length** inputs and/or outputs.

Input–Output Structures

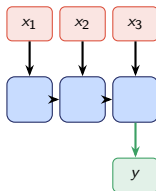
One-to-One



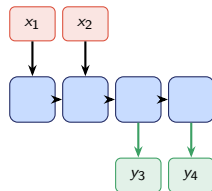
One-to-Many



Many-to-One



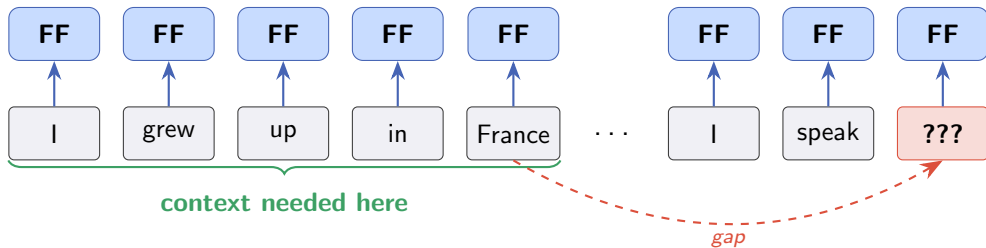
Many-to-Many



RNNs naturally support all of these patterns—the same architecture adapts to different task shapes by choosing *where* to read inputs and *where* to produce outputs.

Feedforward Networks Cannot Remember

No information flows between steps



A feedforward network processes each input **independently**—it has no mechanism to carry “France” forward to predict “French.”

Naive Fix: Sliding Window

One idea: feed the model a fixed window of recent inputs, e.g. the last k tokens.

Window of size 3

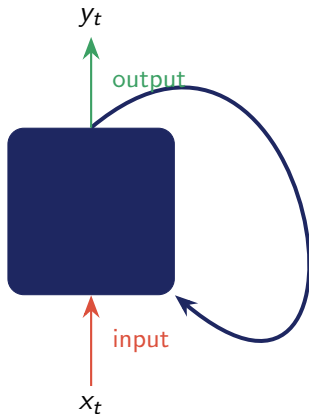
At step t : input = (x_{t-2}, x_{t-1}, x_t) concatenated into one vector.

Problems:

1. **Fixed context:** Cannot look further back than k steps.
2. **Parameter explosion:** Input dimension scales as $k \times d$.
3. **No parameter sharing:** Position 1 and position 2 use different weights, so the model must re-learn patterns at each position.

We need a model that can handle **arbitrary-length** context with a **fixed** number of parameters.

The Core Idea: A Loop



$$h_t = f(h_{t-1}, x_t)$$

hidden state
 h_t

The hidden state h_t is the network's **memory**: it summarizes everything seen so far.

The Vanilla RNN Equations

At each time step t , compute:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

$$y_t = W_y h_t + b_y$$

Parameters (shared across all t):

W_h hidden-to-hidden weights
($n \times n$)

W_x input-to-hidden weights
($n \times d$)

W_y hidden-to-output weights
($m \times n$)

Dimensions:

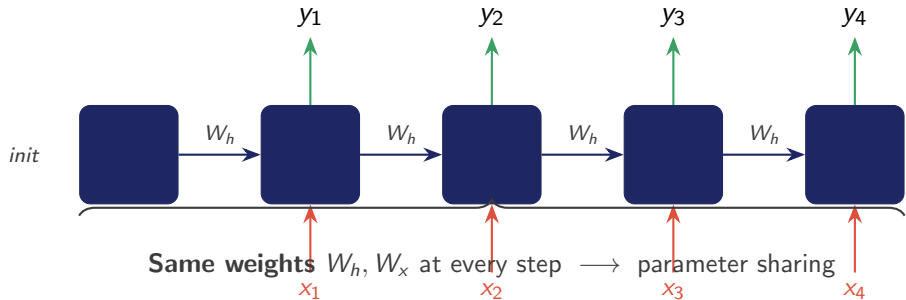
$x_t \in \mathbb{R}^d$ input vector

$h_t \in \mathbb{R}^n$ hidden state

$y_t \in \mathbb{R}^m$ output

The tanh squashes values to $[-1, 1]$, keeping the hidden state bounded.

Unrolling the Loop Through Time



Unrolling reveals an RNN as a very deep network whose **depth equals the sequence length**.

RNN as a Dynamical System

We can view the RNN through the lens of **dynamical systems theory**:

$$h_t = f(h_{t-1}, x_t) \quad \longleftrightarrow \quad s_{t+1} = g(s_t, u_t)$$

RNN

h_t = state (hidden)

x_t = input (observation)

f = learned transition

Control system

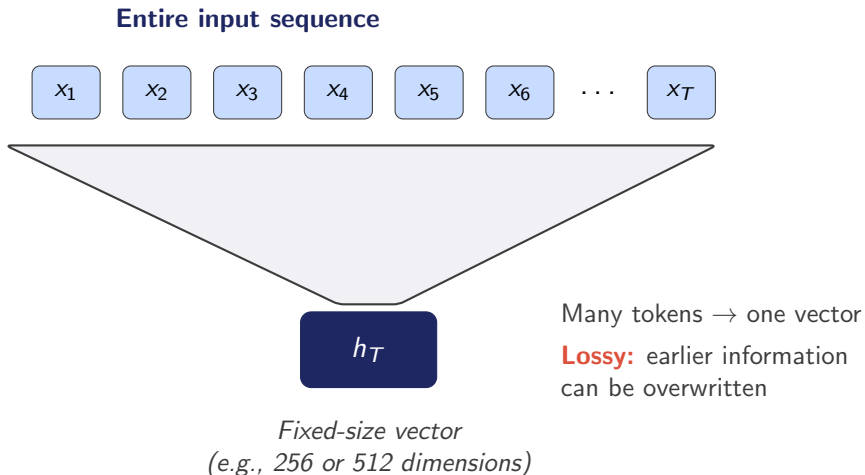
s_t = system state

u_t = control input

g = dynamics function

The network learns *how to update its internal state* given new observations. The quality of the model depends on whether the learned dynamics capture the relevant structure of the data.

RNN as Lossy Compression



A Concrete Example: Character-Level Language Model

Task: Given previous characters, predict the next one.

Training sequence: ‘ ‘hello’ ’

Step	Input x_t	Target y_t	Hidden state	What h_t encodes
1	h	e	h_1	“saw h”
2	e	l	h_2	“saw he”
3	l	l	h_3	“saw hel”
4	l	o	h_4	“saw hell”

At each step the model outputs a probability distribution over the vocabulary (all possible characters) and we train using cross-entropy loss against the true next character.

Training: Backpropagation Through Time (BPTT)

Since the unrolled RNN is just a deep network, we train it with standard backpropagation—but through **time steps** rather than layers.

Algorithm:

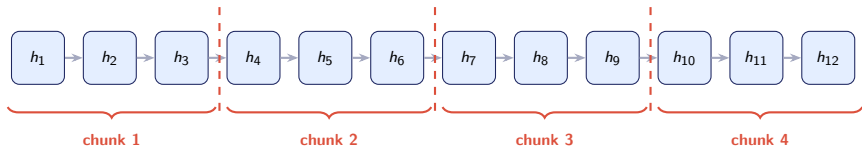
1. **Forward pass:** Run the RNN for T steps, computing $h_1, h_2, \dots, h_T$ and the loss $\mathcal{L} = \sum_{t=1}^T \ell(y_t, \hat{y}_t)$.
2. **Backward pass:** Compute $\frac{\partial \mathcal{L}}{\partial W}$ by unrolling the chain rule through all time steps.
3. **Update:** Apply gradient descent as usual.

Key cost

Memory and computation grow **linearly** with sequence length T , since we must store all intermediate hidden states for the backward pass.

Truncated BPTT

For long sequences, full BPTT is impractical. A common approximation:



Idea: Split the sequence into chunks of length k . Backpropagate gradients only within each chunk.

Trade-off:

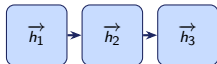
k small $\rightarrow$ fast, but poor long-range learning

k large $\rightarrow$ better gradients, but slower

The forward pass still carries h_t across chunk boundaries (the hidden state is *not* reset), so context is not lost entirely—only gradient information is truncated.

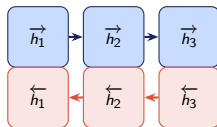
Types of RNN Architectures

Unidirectional



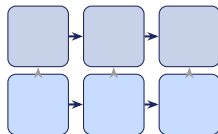
Unidirectional: Only sees past context. Used when future is unavailable (e.g., generation).

Bidirectional



Bidirectional: Sees both past and future. Used when full sequence is available (e.g., classification).

Deep (Stacked)



Deep/Stacked: Multiple RNN layers. Higher layers learn more abstract representations.

Bidirectional RNN: Why and When

Many tasks have access to the **entire** input before producing output.

Named Entity Recognition

“**Paris** is the capital of France” — is “Paris” a city or a person’s name?

You need *both* left context (start of sentence) and right context (“capital of France”) to decide.

A bidirectional RNN runs two passes:

$$\vec{h}_t = f(\vec{h}_{t-1}, x_t) \qquad \overleftarrow{h}_t = f(\overleftarrow{h}_{t+1}, x_t)$$

$$h_t = [\vec{h}_t ; \overleftarrow{h}_t] \quad (\text{concatenation})$$

Cannot be used for autoregressive generation (would require seeing the future).

Deep RNNs: Stacking Layers

A single-layer RNN may not have enough capacity. We can **stack** multiple layers:

$$h_t^{(l)} = f(h_{t-1}^{(l)}, h_t^{(l-1)}) \quad l = 1, 2, \dots, L$$

where $h_t^{(0)} = x_t$ is the input and $h_t^{(L)}$ is the final representation.

Intuition: Each layer learns features at a different level of abstraction.

Layer 1

Low-level patterns: n-grams, local dependencies

Layer 2+

Higher-level: phrases, long-range structure, semantics

In practice, 2–4 layers are common. Beyond that, training becomes difficult without residual connections.

Memory Gets Overwritten at Every Step



Memory content:



h_1 : fresh



h_2 : mixed



h_3 : diluted

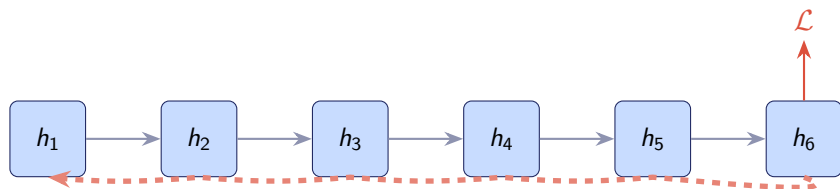


h_4 : faded

$$h_t = \tanh(W h_{t-1} + U x_t) \quad \leftarrow \text{full overwrite, not selective update}$$

Like a game of *telephone*: each retelling distorts the original message a little more.

The Gradient Must Travel Back in Time



$\times W^T$

$\times W^T$

$\times W^T$

$\times W^T$

$\times W^T$

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_6} \cdot \prod_{i=2}^6 \frac{\partial h_i}{\partial h_{i-1}}$$

Each factor $\approx W^T \cdot \text{diag}(\tanh')$
Product of many terms!

Why Products of Matrices Are Dangerous

Consider the Jacobian of one step:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\tanh'(z_t)) \cdot W_h$$

Over k steps, the gradient involves a **product of k matrices**:

$$\prod_{i=t-k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \approx (D \cdot W_h)^k$$

The behaviour of this product is governed by the **eigenvalues** λ of W_h :

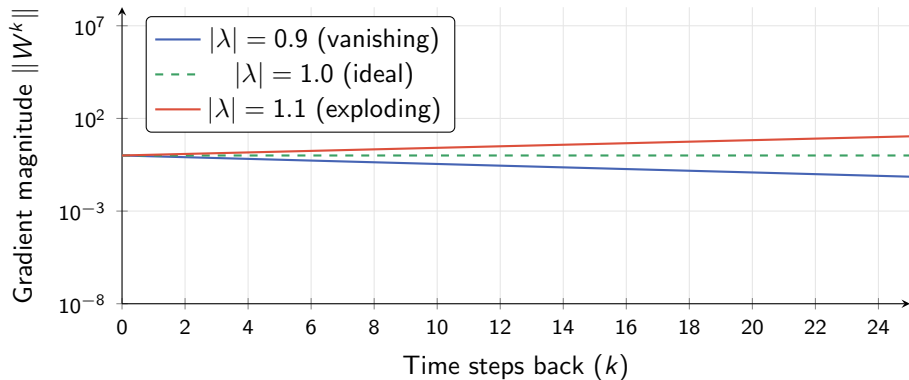
$$\begin{aligned} |\lambda| &< 1 \\ |\lambda|^k &\rightarrow 0 \\ \text{Vanishing} \end{aligned}$$

$$\begin{aligned} |\lambda| &= 1 \\ |\lambda|^k &= 1 \\ \text{Ideal (but unstable)} \end{aligned}$$

$$\begin{aligned} |\lambda| &> 1 \\ |\lambda|^k &\rightarrow \infty \\ \text{Exploding} \end{aligned}$$

Even $|\lambda| = 0.99$ gives $0.99^{100} \approx 0.37$ and $0.99^{500} \approx 0.007$. The gradient effectively disappears.

Vanishing vs. Exploding Gradients



Even small deviations from $|\lambda| = 1$ compound exponentially over many steps. This is why vanilla RNNs struggle with sequences longer than ~ 10 – 20 tokens.

Practical Consequence: Short-Term Memory Only

Experiment (Bengio et al., 1994)

Train an RNN to detect whether a specific token appeared k steps ago.

Result: performance degrades rapidly for $k > 10$.

What this means in practice:

An RNN trained on “I grew up in France. I went to school in Paris. I studied literature. Eventually I moved abroad. After many years I returned. I speak fluent _____”

will likely *fail* to predict “French” because “France” appeared too many steps ago. The gradient signal from the loss at the end cannot effectively reach the parameters that process “France.”

The fundamental tension

We need long-range memory, but gradient-based training makes it nearly impossible for vanilla RNNs to learn it.

Gradient Clipping: A Partial Fix

Exploding gradients can be mitigated with a simple heuristic:

$$\hat{g} = \begin{cases} g & \text{if } \|g\| \leq \theta \\ \frac{\theta}{\|g\|} \cdot g & \text{if } \|g\| > \theta \end{cases}$$

This rescales the gradient to have norm at most θ whenever it exceeds the threshold.

What it fixes

Prevents numerical overflow and training instability caused by exploding gradients.

What it does NOT fix

Vanishing gradients are the deeper problem—clipping cannot *amplify* a signal that has already disappeared.

Key Insight: Don't Overwrite—Gate



Full overwrite
every step

Selective update
forget old / add new

Forget gate f_t

What to *erase* from memory

Input gate i_t

What *new* info to store

Output gate o_t

What to *expose* as output

LSTM Equations

Given inputs x_t , previous hidden state h_{t-1} , and previous cell state c_{t-1} :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{candidate memory})$$

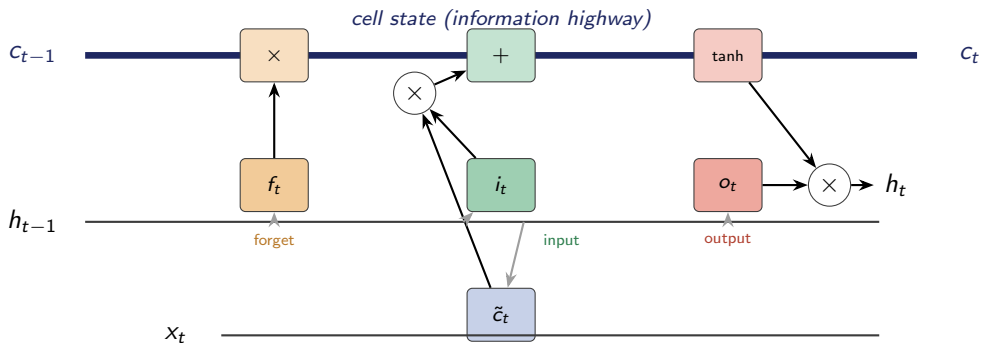
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cell state update})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate})$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{hidden state})$$

Here σ is the sigmoid function ($\rightarrow [0, 1]$), $\odot$ is element-wise multiplication, and $[h_{t-1}, x_t]$ denotes concatenation.

Inside an LSTM Cell



Intuition for Each Gate

Think of the LSTM as a **controlled notebook**:

Forget gate f_t

“Which parts of my notes are no longer relevant?”

$f_t = 0$: erase completely

$f_t = 1$: keep everything

Input gate i_t

“Which parts of this new observation are worth writing down?”

$i_t = 0$: ignore this input

$i_t = 1$: write it all

Output gate o_t

“Which parts of my notes are relevant for answering *right now*?”

$o_t = 0$: stay silent

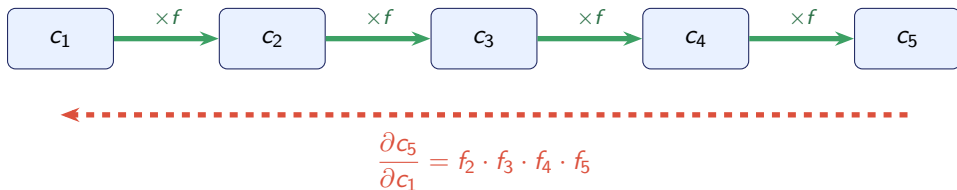
$o_t = 1$: expose all

Concrete scenario

Reading “The cat, which was adopted from a shelter last year, sat on the mat.”

The forget gate might learn to maintain subject information (“cat”) across the relative clause, and the input gate might learn to suppress storing the parenthetical details.

Why LSTM Solves the Gradient Problem



When $f_t \approx 1$: gradient flows **unchanged**—no vanishing!

The cell state acts as an **information highway** with learned on-ramps and off-ramps.

LSTM vs. Vanilla RNN: Gradient Comparison

Vanilla RNN gradient

$$\prod_{i=t-k+1}^t \text{diag}(\tanh') \cdot W_h$$

Many non-linear, *uncontrolled* factors multiply together. $\tanh' \in (0, 1]$ further shrinks the signal.

LSTM cell-state gradient

$$\prod_{i=t-k+1}^t f_i$$

A product of *learned* scalars. The network can set $f_i \approx 1$ to let gradients through, or $f_i \approx 0$ to “close the gate.”

The critical difference: in an LSTM, the network **chooses** how much gradient to propagate. In a vanilla RNN, the gradient decays passively through fixed non-linearities.

Forget Gate Bias Initialization

A practical but important detail: the forget gate bias is typically initialized to a **large positive value** (e.g., $b_f = 1$ or $b_f = 2$).

Why?

At the start of training, with random weights:

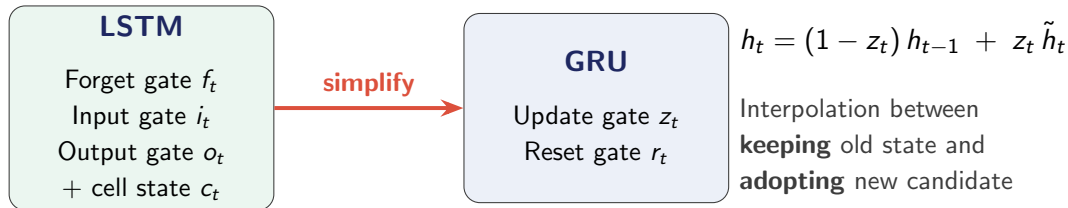
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

If $b_f = 0$, the sigmoid output is ≈ 0.5 on average, meaning the cell state is *halved* at every step. Over 10 steps: $0.5^{10} \approx 0.001$ —the cell state vanishes before learning even begins.

With $b_f = 1$: $\sigma(1) \approx 0.73$, giving the network a fighting chance to preserve information early in training.

This was identified by Jozefowicz et al. (2015) as one of the most important hyperparameters for LSTM performance.

GRU: Two Gates Instead of Three



Fewer parameters, often comparable performance. A good default when LSTM feels like overkill.

GRU Equations in Detail

The GRU merges the forget and input gates into a single **update gate**:

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (\text{update gate: how much to update})$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (\text{reset gate: how much past to use})$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t]) \quad (\text{candidate hidden state})$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{final state: interpolation})$$

Key simplifications vs. LSTM:

1. No separate cell state—the hidden state h_t serves both roles.
2. The forget and input amounts are **coupled**: $1 - z_t$ and z_t always sum to 1.
3. The reset gate r_t controls how much of h_{t-1} is used to compute the candidate.

LSTM vs. GRU: When to Use Which?

	LSTM	GRU
Parameters	More (4 gates)	Fewer (2 gates)
Training speed	Slower	Faster
Cell state	Separate c_t	Merged into h_t
Gate coupling	Independent	Forget/input tied
Long sequences	Often slightly better	Competitive
Small datasets	Can overfit more	Better regularized

Rule of thumb: Start with GRU; switch to LSTM if you need the extra capacity or if the task involves very long-range dependencies. In practice, the difference is often small.

Empirical comparison: Chung et al. (2014) found neither consistently outperforms the other across tasks.

Regularization for RNNs

Standard dropout doesn't work well on recurrent connections (it disrupts the memory).

Effective techniques:

- 1. Dropout on non-recurrent connections only.** Apply dropout to the input x_t and the output $h_t \rightarrow y_t$, but *not* to the $h_{t-1} \rightarrow h_t$ connection.
- 2. Variational dropout** (Gal & Ghahramani, 2016). Use the *same* dropout mask at every time step. This is equivalent to a Bayesian approximation and preserves the memory signal.
- 3. Weight tying.** In language models, share the embedding matrix and the output projection matrix. Reduces parameters significantly.
- 4. Zoneout** (Krueger et al., 2017). Randomly keep the previous hidden state instead of updating—a dropout variant specifically designed for recurrent units.

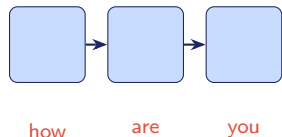
Common Implementation Pitfalls

- 1. Packing and padding.** Sequences in a batch usually have different lengths. Use padding + masking or packed sequences (PyTorch: `pack_padded_sequence`). Forgetting to mask padded positions corrupts the hidden state.
- 2. Hidden state initialization.** h_0 is typically initialized to zeros, but in some tasks learning h_0 as a parameter improves performance.
- 3. Gradient clipping threshold.** A common default is $\theta = 5.0$. Monitor the gradient norm during training—if it's frequently clipped, the learning rate may be too large.
- 4. Teacher forcing.** During training, feed the *ground truth* y_{t-1} as input to step t (not the model's own prediction). At inference time, the model must use its own outputs—this discrepancy (*exposure bias*) can degrade generation quality.

Sequence-to-Sequence (Encoder–Decoder)

One of the most influential RNN applications: map a variable-length input to a variable-length output.

Encoder



Decoder

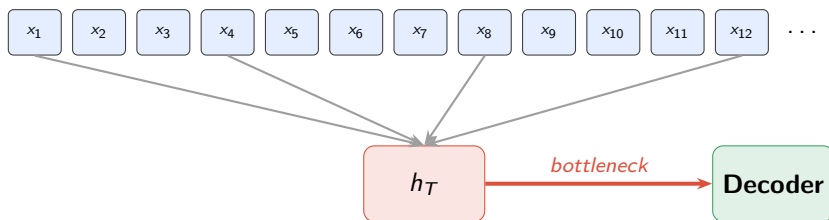
comment allez vous



The encoder reads the input and produces a context vector c (typically h_T or a function of all hidden states). The decoder generates the output one token at a time, conditioned on c .

Applications: machine translation, summarization, dialogue systems, code generation.

The Fundamental Bottleneck



Problem: 500 tokens squeezed into one vector.
The decoder cannot “look back” at individual tokens.

Attention: A Sketch of the Solution

Instead of compressing the entire input into one vector, let the decoder **look at all encoder states**:

$$\alpha_{t,i} = \frac{\exp(e(h_t^{\text{dec}}, h_i^{\text{enc}}))}{\sum_j \exp(e(h_t^{\text{dec}}, h_j^{\text{enc}}))} \quad c_t = \sum_i \alpha_{t,i} h_i^{\text{enc}}$$

$\alpha_{t,i}$ are **attention weights**—a learned soft alignment between decoder step t and encoder position i .

The context vector c_t is now *different at every decoder step*, focusing on the most relevant parts of the input.

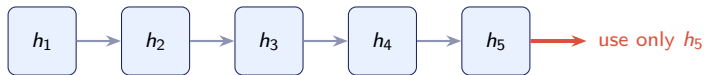
Key benefit

No more fixed-size bottleneck. The decoder can retrieve any information from the encoder, regardless of distance.

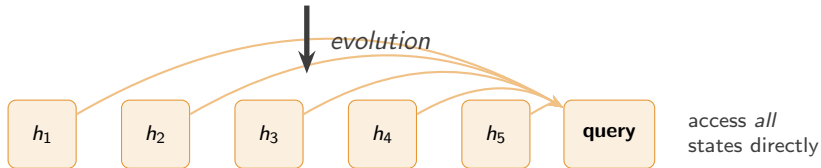
Introduced by Bahdanau et al. (2015). This idea eventually evolved into the Transformer (Vaswani et al., 2017), which replaces recurrence entirely with self-attention.

From Compression to Direct Access

RNN:



Attention:



This insight—**let the decoder attend to all encoder states**—led first to attention mechanisms and then to the Transformer architecture, which removes recurrence entirely.

RNNs Are Not Dead

Transformers dominate NLP today, but RNNs remain relevant in several settings:

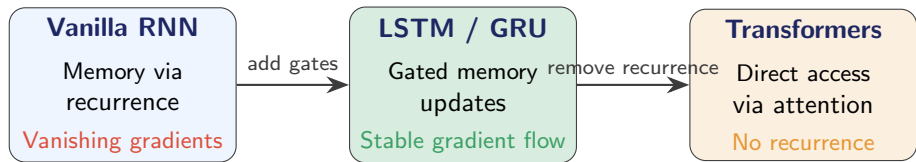
Where RNNs still shine:

1. **Edge/embedded devices:** LSTM and GRU have small memory footprint and can process streams token-by-token without storing the full sequence.
2. **Online/streaming tasks:** Real-time speech recognition, sensor data, robotics—where data arrives continuously and decisions must be made immediately.
3. **Small data regimes:** With fewer parameters than Transformers, RNNs can be easier to train on limited data.

Recent resurgence:

1. **State-space models** (S4, Mamba) borrow ideas from both RNNs and linear systems, achieving Transformer-level performance with RNN-like inference.
2. These can be viewed as “RNNs done right”—recurrence with carefully designed dynamics.

The Evolution of Sequence Models



Each step solved a specific limitation of the previous architecture.

Understanding **why** each model fails helps you understand **why** the next one exists.

Key Takeaways

- 1. Sequences need memory.** Feedforward networks process inputs independently; RNNs introduce a hidden state that carries information across time steps.
- 2. Vanilla RNNs have a fatal flaw.** Repeated matrix multiplication causes gradients to vanish (or explode), making it nearly impossible to learn long-range dependencies.
- 3. Gating is the key insight.** LSTM and GRU add learnable gates that control memory flow, creating a gradient highway through the cell state.
- 4. The bottleneck problem persists.** Even gated RNNs compress all history into a single vector, which limits capacity for very long sequences.
- 5. Attention removes the bottleneck.** By letting the decoder look at all encoder states, attention mechanisms eliminate the fixed-size compression, paving the way for Transformers.

References and Further Reading

Foundational papers:

Elman (1990) “Finding Structure in Time” — the original simple RNN.

Hochreiter & Schmidhuber (1997) “Long Short-Term Memory” — the LSTM.

Cho et al. (2014) “Learning Phrase Representations...” — the GRU.

Bahdanau et al. (2015) “Neural Machine Translation by Jointly Learning to Align and Translate” — attention.

Practical guides:

Karpathy (2015) “The Unreasonable Effectiveness of Recurrent Neural Networks” — intuitive blog post with code.

Olah (2015) “Understanding LSTM Networks” — the classic visual explainer.

Modern connections:

Gu et al. (2022) “Efficiently Modeling Long Sequences with Structured State Spaces” — S4 / state-space models.